

## Compte rendu des commandes Git

### 1. ``ls``

Cette commande permet d'afficher le contenu d'un dossier. Elle est utilisée ici pour vérifier la présence du dépôt avant de le cloner.

### 2. ``git clone <source> <destination>``

Cette commande permet de copier un dépôt Git existant dans un nouveau répertoire. Le dépôt cloné est une copie complète qui reste liée au dépôt d'origine, appelé dépôt distant (\*remote\*).

### 3. ``git status``

Elle permet de connaître l'état du dépôt local :

- \* la branche utilisée,
- \* si le dépôt est à jour avec le dépôt distant,
- \* s'il y a des modifications non enregistrées.

### 4. ``git remote -v``

Cette commande affiche la liste des dépôts distants associés au dépôt local.

Elle indique aussi les adresses utilisées pour récupérer (\*fetch\*) et envoyer (\*push\*) des modifications.

### 5. ``git config --global user.email "email"``

Permet de définir l'adresse de courriel associée aux commits pour tous les dépôts Git de l'utilisateur.

Cette adresse est utilisée pour identifier l'auteur des commits sur GitHub.

### 6. ``git config --global user.email``

Affiche l'adresse de courriel définie dans la configuration globale de Git.

### 7. ``git config user.email "email"``

Permet de définir une adresse de courriel spécifique pour un dépôt précis, sans modifier la configuration globale.

## 8. ``git config user.email``

Affiche l'adresse de courriel configurée pour le dépôt courant.

## 9. ``git clone https://github.com/USERNAME/REPO.git``

Cette commande permet de cloner un dépôt hébergé sur GitHub via Internet.

Lors de l'opération, l'utilisateur doit entrer son nom d'utilisateur GitHub et un **\*\*jeton d'accès personnel\*\***, qui remplace le mot de passe.

## 10. Authentification avec jeton d'accès personnel

Lors des commandes Git utilisant HTTPS (comme ``git clone`` ou ``git push``), Git demande :

- \* le nom d'utilisateur GitHub,
- \* le **\*\*Personal Access Token\*\*** à la place du mot de passe.

Ce jeton permet d'assurer la sécurité des accès au compte GitHub.

## 11. Conclusion

Les commandes Git présentées permettent de :

- \* cloner des dépôts locaux ou distants,
- \* vérifier l'état d'un dépôt,
- \* gérer les liens avec les dépôts distants,
- \* configurer l'identité de l'utilisateur,
- \* sécuriser l'accès à GitHub.

Elles constituent la base nécessaire pour travailler avec Git et GitHub en ligne de commande.

Se placer dans le dossier parent du dépôt et vérifier son contenu avec :

```
$ ls
```

GITTP1

Cloner le dépôt avec :

```
$ git clone GITTP1 GITTP2
```

Cela crée un nouveau dossier

Vérifier l'état du dépôt et la liaison avec le dépôt distant :

```
$ cd GITTP2
```

Compte GitHub et configuration de l'adresse e-mail

Créer un compte sur GitHub

si ce n'est pas déjà fait.

GitHub associe les commits à votre compte via l'adresse de commit. Pour protéger sa vie privée, GitHub fournit une adresse noreply que l'on peut utiliser :

<ID+USERNAME>@users.noreply.github.com.

Configuration dans Git :

Adresse globale (tous les dépôts) :

```
$ git config --global user.email "<ID+USERNAME>@users.noreply.github.com"
```

```
19gem@DESKTOP-KGNLVPV MINGW64 ~/Desktop
$ git clone GITTP1 GITTP2
Cloning into 'GITTP2'...
warning: You appear to have cloned an empty repository.
done.

19gem@DESKTOP-KGNLVPV MINGW64 ~/Desktop
$ cd GITTP2

19gem@DESKTOP-KGNLVPV MINGW64 ~/Desktop/GITTP2 (main)
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

19gem@DESKTOP-KGNLVPV MINGW64 ~/Desktop/GITTP2 (main)
$ git config --global user.email manuel.mounianman1@gmail.com

19gem@DESKTOP-KGNLVPV MINGW64 ~/Desktop/GITTP2 (main)
$ git config user.email
manuel.mounianman1@gmail.com

19gem@DESKTOP-KGNLVPV MINGW64 ~/Desktop/GITTP2 (main)
```

Adresse locale (dépôt spécifique) :

```
$ git config user.email "<ID+USERNAME>@users.noreply.github.com"
```

Jeton d'accès personnel (Personal Access Token)

GitHub ne permet plus de se connecter par mot de passe pour Git HTTPS. Il faut générer un Personal Access Token (classic) dans les Developer settings.

Paramétrage : nom du jeton, durée de validité (ex.

```
19gem@DESKTOP-KGNLVPU MINGW64 ~/desktop/GITTP2 (main)
$ git clone https://github.com/manuelmmn-svg/GITTP2.git
Cloning into 'GITTP2'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

19gem@DESKTOP-KGNLVPU MINGW64 ~/desktop/GITTP2 (main)
$ |
```

semestre), permissions : repo.

Utilisation du jeton :

\$

Username: <YOUR\_USERNAME>

Password: <YOUR\_TOKEN>

Le jeton peut être mis en cache pour éviter de le ressaisir à chaque opération.

Mise en pratique

Création du dépôt local :

```
$ mkdir mon-projet
```

```
$ cd mon-projet
```

```
$ git init
```

Lien avec le dépôt distant GitHub :

```
$ git remote add origin https://github.com/<USERNAME>/<REPO>.git
```

Création du fichier README.md :

```
$ echo "# Mon projet" > README.md
```

```
$ git add README.md
```

```
$ git commit -m "Ajout du README"
```

**\$ git push -u origin main**

**Création d'une nouvelle branche :**

**\$ git branch nouvelle-branche**

**\$ git checkout nouvelle-branche**

**Modification et validation :**

**\$ echo "Ajout d'informations" >> README.md**

**\$ git add README.md**

**\$ git commit -m "Mise à jour du README"**

**Fusion avec la branche principale et push :**

**\$ git checkout main**

**\$ git merge nouvelle-branche**

**\$ git push**

**Ajout du compte-rendu et push final :**

**\$ git add compte-rendu.pdf**

**\$ git commit -m "Ajout du compte-rendu"**

**\$ git push**

**Conclusion**

**Ce TP permet de maîtriser les étapes essentielles de Git et GitHub : créer et cloner des dépôts, configurer son identité et sa sécurité, utiliser des branches pour développer et fusionner des modifications, publier et synchroniser les fichiers avec un dépôt distant. L'ensemble des manipulations assure la traçabilité des modifications, la collaboration et la sécurité lors de l'utilisation de Git et GitHub**